

Durum Vektörü ve Durum Uzayı Yaklaşımı

ÖFY

October 2026

Özet

Durum vektörü (*state vector*), bir sistemin belirli bir andaki halini tam olarak tarif eden sayıların listesidir; değeri, geçmişin tamamını bilmeden sistemin bir sonraki halini hesaplamaya yetmesinden gelir. Zamanın üç yönü üç ayrı işlemle karşılanır: geçmiş saklanır (log), şimdi özetlenir (durum vektörü), gelecek hesaplanır (model). Log, durumdan daha fazla bilgi taşır; durum logdan her zaman türetilabilir, ancak log durumdan geri çıkarılamaz. Durum vektörü hazır gelmez; ham gözlemden durum çıkarımı (*state extraction*) ile üretilir ve iyi bir çıkarımın ölçütü, vektörün küçük kalırken bir sonraki adım için gereken her şeyi içermesidir. Bu kavramların ortak çatısı durum uzayı yaklaşımıdır (*state-space approach*); bu yaklaşım da zamanla değişen sistemlerin uzun vadeli davranışını inceleyen dinamik sistemler teorisinin bir parçasıdır. Yazılımda aynı fikirler farklı adlarla zaten kullanılmaktadır: **reducer**, durum makinesi, *event sourcing*, *snapshot*, *optimistic update*. Geri beslemeli altyapı mekanizmaları (otomatik ölçekleme, yeniden deneme) birer dinamik sistemdir ve kararlılık sorunları bu gözle okunduğunda anlaşılır hale gelir.

Giriş

Zamanla değişen bir sistemi anlamak isteyen herkes aynı soruyla karşılaşır: sistemin geleceğini söyleyebilmek için ne kadar bilgi gerekir? Hareket eden bir arabanın yalnızca konumunu bilmek yetmez, çünkü konum arabanın nereye gittiğini söylemez. Öte yandan arabanın yola çıktığı andan beri yaptığı her hareketi bilmek de gerekmez. Gereken, ikisinin arasında bir yerdedir: şu anki konum ve şu anki hız.

Bu “gereken en az bilgi” fikrinin adı **durum**dur. Durumu sayılarla yazıp bir listeye koyunca **durum vektörü** elde edilir. Durum vektörünün alabileceği tüm değerlerin kümesine **durum uzayı** (*state space*), durumun bir andan sonrakine nasıl geçtiğini söyleyen kurala **geçiş modeli**, durumların zaman içindeki sırasına **yörünge** (*trajectory*) denir.

Yaygın bir yanlış, “durum vektörü” teriminin tek bir alana ait olduğunu düşünmektir. Terim kuantum mekaniğinde, kontrol teorisinde, yörünge mekaniğinde, dağıtık sistemlerde ve makine öğrenmesinde geçer ve ilk bakışta bunlar ilgisiz kullanımlar gibi görünür. Oysa hepsinde aynı fikir vardır: geçmişin tamamını taşımak yerine, devam etmek için gereken bilgiyi tek bir vektörde tutmak.

Bu not kavramların tanımlarını, aralarındaki ilişkiyi ve yazılımdaki karşılıklarını kapsar. Matematiksel ayrıntılar (denklemlerin çözümü, kararlılık ispatları, filtre türetimleri) kapsam dışındadır.

Durum Vektörü

Tanım

Durum vektörü, bir sistemin belirli bir andaki halini tam olarak tarif eden sayıların listesidir. “Tam” kelimesi burada belirli bir anlam taşır: bu sayılar ve sistemin kuralı bilindiğinde, sistemin bir sonraki hali hesaplanabilir. Geçmişe ayrıca bakmaya gerek kalmaz.

Araba örneğinde durum vektörü şöyledir:

$$x = [konum, hız]$$

Yalnızca konum bir durum vektörü değildir, çünkü aynı konumdaki iki araba farklı hızlarda olabilir ve bir saniye sonra farklı yerlerde bulunur. Konum ve hız birlikte verildiğinde ise bir sonraki an belirlenir.

Alanlara Göre Kullanımı

Terim farklı alanlarda aynı fikri taşır.

- **Kontrol teorisi ve fizik.** Sistemin değişkenleri (konum, hız, sıcaklık, akım) tek bir vektörde toplanır. Sistem $\dot{x} = Ax + Bu$ biçiminde bir denklemlerle ilerler; burada x durum vektörü, u dışarıdan verilen girdidir.
- **Kuantum mekaniği.** Bir kuantum sisteminin durumunu veren vektördür ve $|\psi\rangle$ biçiminde yazılır. İçindeki sayılar, ölçüm yapıldığında her sonucun çıkma olasılığını belirler.
- **Yörünge mekaniği.** Bir uydunun üç konum ve üç hız bileşeninden oluşan altı sayılık vektörüdür.
- **Dağıtık sistemler.** Çakışmasız çoğaltılmış veri tiplerinde (*Conflict-free Replicated Data Type*, CRDT) durum vektörü, her istemciden o ana kadar kaç güncelleme görüldüğünü tutan bir tablodur. İki taraf tablolarını karşılaştırır ve yalnızca eksik güncellemeler gönderilir (Shapiro vd., 2011).
- **Makine öğrenmesi.** Bir modelin o ana kadar gördüklerini özetleyen iç vektördür. Yinelemeli sinir ağlarındaki (*Recurrent Neural Network*, RNN) gizli durum (*hidden state*) buna örnektir.

Geçmiş, Şimdi ve Gelecek

Geçmiş: Log

Geçmişin tamamı saklanmak istendiğinde durum vektörü yerine bir **log** (kayıt geçmiş) tutulur: olan her şey sırayla yazılır ve hiçbir şey silinmez.

İkisi farklı sorulara cevap verir. Durum vektörü “şu an neredeyim” sorusunun cevabıdır; küçüktür ve boyutu sabittir. Log “buraya nasıl geldim” sorusunun cevabıdır; zamanla sürekli büyür.

Banka hesabı bu ayrımı açık biçimde gösterir. Bakiye durumdur, hesap hareketleri logdur. Hareketlerden bakiye her zaman hesaplanabilir, ancak bakiyeden hareketler geri çıkarılamaz. Log bu nedenle daha fazla bilgi taşır; durum onun özetidir.

Bu yaklaşımın alanlara göre adları şunlardır:

- **Yazılım.** *Event sourcing*: veritabanında son hal değil, olaylar saklanır; son hal olaylar baştan oynatılarak bulunur (Fowler, 2005). Sürüm kontrol sistemlerindeki **commit** geçmiş ve veritabanlarındaki *write-ahead log* aynı fikre dayanır (Kleppmann, 2017).
- **Fizik ve kontrol.** Yörünge, yani durum vektörünün her andaki değerlerinin dizisi.
- **CRDT.** Güncelleme geçmişinin tamamı. Durum vektörü burada yalnızca “geçmişin neresine kadar görüldü” bilgisini tutar.
- **Makine öğrenmesi.** *Transformer* mimarisi tek bir özet vektöre dayanmak yerine önceki girdilerin tümüne bakar (Vaswani vd., 2017).

Logun iki bedeli vardır: depolama büyür ve güncel durumu bulmak için geçmiş baştan oynatmak yavaşlar. Bu nedenle pratikte ikisi birlikte kullanılır. Log tutulur, belirli aralıklarla da durumun bir kopyası alınır; buna **snapshot** denir. Böylece oynatma baştan değil, son *snapshot*’tan başlar.

Gelecek: Tahmin

Gelecek saklanamaz, çünkü henüz gerçekleşmemiştir; yalnızca hesaplanabilir. Hesap için iki şey gerekir: şu anki durum ve geçiş modeli.

$$x_{t+1} = f(x_t, u_t)$$

Bu işlem tekrar tekrar uygulandığında geleceğe doğru bir durum dizisi elde edilir. Buna **tahmin** (*prediction*) ya da **simülasyon** denir. Durum vektörünün asıl faydası burada ortaya çıkar: geçmiş bilinmeden, yalnızca şimdiki durumdan ileri hesaplanabilir.

Geleceğin ne kadar kesin olduğu sistemin türüne bağlıdır.

- **Deterministik sistem.** Kural kesindir ve tek bir gelecek vardır. Bir uydunun konumu ve hızı biliniyorsa yıllar sonraki yeri hesaplanır.

- **Stokastik sistem.** Kuralda rastgelelik vardır; tek bir gelecek yerine olasılıklar elde edilir. Hava tahmini böyledir ve ileriye gidildikçe belirsizlik büyür.

Tahminin alanlara göre karşılıkları:

- **Kontrol.** Model öngörülü kontrolde (*Model Predictive Control*) sistem birkaç adım ilerisini simüle eder, en iyi sonucu veren hamleyi seçer ve bir sonraki adımda hesabı yeniler.
- **Kalman filtresi.** Önce durum ileri tahmin edilir, sonra gelen ölçümle düzeltilir (Kalman, 1960).
- **Kuantum mekaniği.** Schrödinger denklemi durum vektörünü zamanda ileri taşır. Vektörün gelişimi kesindir, ölçüm sonucu ise olasılıklıdır.
- **Makine öğrenmesi.** Dil modelleri bir sonraki kelimeyi tahmin eder, onu girdiye ekler ve devam eder.
- **Yazılım.** Bilinen gelecek plan olarak tutulur (zamanlanmış işler, kuyruklar). *Optimistic update* küçük bir gelecek tahminidir: sonucu cevabı beklenmeden sonuç varsayıp ekrana yansıtılır, yanlış çıkarsa geri alınır.

Üç Yönün Özeti

Zaman	İşlem	Araç
Geçmiş	Saklanır	Log
Şimdi	Özetlenir	Durum vektörü
Gelecek	Hesaplanır	Geçiş modeli

Durum Çıkarımı

Tanım

Durum çıkarımı (*state extraction*), ham veriden durum vektörünü üretme işlemidir. “Özelliklerini çıkar” ifadesi bunu anlatır: dağınık veriden, durumu tarif eden önemli sayılar seçilir ve bir vektöre konur.

Gerçek sistemlerde durum vektörü hazır gelmez. Eldeki çoğu zaman ham veridir: kamera görüntüsü, sensör okumaları, metin, log kayıtları. Bu veri büyük ve gürültülüdür; büyük kısmı karar için işe yaramaz. Durum çıkarımı bu veriden yalnızca gerekeni alır. Kontrol teorisinde aynı işleme durum kestirimi (*state estimation*) denir.

Örnekler

- **Araba.** Kamera görüntüsü milyonlarca pikselden oluşur. Buradan konum, hız, şeride uzaklık ve öndeki araca mesafe çıkarılır. Karar vermek için bu dört sayı yeter.
- **Satranç.** Tahtanın fotoğrafından taşların hangi karede olduğu çıkarılır.
- **Ses.** Ham dalga formundan frekans, ses yüksekliği ve tempo çıkarılır.
- **Sohbet.** Uzun bir konuşmadan kullanıcının ne istediği, hangi bilgilerin verildiği ve neyin eksik olduğu çıkarılır.

Yöntemler

Çıkarma iki yolla yapılır.

- **Elle.** Hangi özelliklerin önemli olduğuna tasarımcı karar verir ve bunları hesaplayan kod yazılır.
- **Öğrenerek.** Bir sinir ağı (kodlayıcı, *encoder*) hangi özelliklerin önemli olduğunu veriden kendisi öğrenir. Çıkan vektöre *embedding* ya da gizil durum (*latent state*) denir. Durumu ve geçiş kuralları birlikte öğrenen modellere dünya modeli (*world model*) adı verilir (Ha ve Schmidhuber, 2018).

Ölçüt

İyi bir çıkarımın ölçütü iki yönlüdür: vektör küçük olmalı, ancak bir sonraki adımı hesaplamak için gereken her şeyi içermelidir. Eksik bilgi tahmini bozar; arabanın hızı vektörde yoksa bir sonraki konum hesaplanamaz. Fazla bilgi ise sistemi yavaşlatır ve gürültüyü içeri taşır.

Durum Uzayı Yaklaşımı

Tanım

Durum uzayı yaklaşımı, bir sistemi şu yöntemle inceler: sistem bir durum vektörüyle tarif edilir, sonra bu vektörün zamanla nasıl değiştiğini yazan kural bulunur. Üç parçası vardır:

- **Durum.** Sistemin şu anki halini veren sayılar.
- **Girdi.** Sisteme dışarıdan yapılan etki.
- **Kural.** Durum ve girdiden bir sonraki durumu veren denklem.

Araba örneğinde durum konum ve hızdır, girdi gaz pedalıdır, kural ise “hız konumu değiştirir, gaz hızı değiştirir” biçimindedir.

“Uzay” Kelimesinin Anlamı

Durum vektörünün her sayısı bir eksen olarak düşünülür. Araba için bir eksen konum, diğeri hız olur ve iki eksenli bir düzlem ortaya çıkar. Arabanın her olası hali bu düzlemde bir noktadır. Araba hareket ettikçe nokta da hareket eder ve bir çizgi çizer. Düzleme durum uzayı, çizilen çizgiye yörünge denir. Vektörde on sayı varsa uzay on boyutludur; çizilemez ama fikir aynıdır.

Alternatifi: Girdi-Çıktı Yaklaşımı

Daha eski yöntem sisteme kapalı kutu olarak bakar: yalnızca “şu girdi verildiğinde şu çıktı alınır” ilişkisi incelenir, içeride ne olduğuna bakılmaz. Durum uzayı yaklaşımı kutunun içini açar ve iç değişkenleri doğrudan izler. Kontrol mühendisliğinde bu yaklaşımın yaygınlaşması 1960'lara, Kalman'ın çalışmalarına dayanır (Kalman, 1960).

Faydaları

- Çok girdili ve çok çıktıli sistemlerde rahat çalışır.
- Bilgisayarda simüle edilmesi kolaydır, çünkü her adım aynı işlemdir: durum alınır, kural uygulanır, yeni durum yazılır.
- Doğrudan ölçülemeyen iç değişkenler de hesaba katılır.
- Mekanik, elektrik, ekonomi ve yazılım gibi farklı sistemler aynı kalıpla yazılır.

Aynı Çatının Farklı Adları

Alan	Karşılığı
Kontrol teorisi	Durum uzayı gösterimi, durum kestirimi, Kalman filtresi
Pekiştirmeli öğrenme	Markov karar süreci: durum, eylem, geçiş, ödül
Makine öğrenmesi	Durum temsili öğrenme, dünya modelleri
Yazılım	Durum makinesi, <i>event sourcing</i> , durum yönetimi

Pekiştirmeli öğrenmedeki (*reinforcement learning*) Markov karar süreci (*Markov Decision Process*, MDP), durum uzayı yaklaşımına ödül kavramının eklenmiş halidir (Bellman, 1957; Sutton ve Barto, 2018). Bu alanda tipik sıra şöyledir: gözlem alınır, durum çıkarılır, karar verilir, eylem uygulanır.

Dinamik Sistemler Teorisi

Tanım

Dinamik sistemler teorisi, zamanla değişen sistemleri inceleyen matematik dalıdır. “Dinamik” burada “zamanla değişen” anlamına gelir. Sarkaç, gezegenler, nüfus, hava durumu, bir ekonomi ve bir salgın birer dinamik sistemdir.

Bir dinamik sistem iki şeyden oluşur: durum ve durumun zamanla nasıl değiştiğini söyleyen kural. Durum uzayı yaklaşımı bu ikisini yazma yöntemidir. Dinamik sistemler teorisi bir adım ileri gider ve şu soruyu sorar: şu anki durum ve kural biliniyorsa, uzun vadede ne olur?

Temel Sorular

- **Denge.** Sistem bir noktada durur mu? Durgun sarkaç bir dengedir.
- **Kararlılık.** Sistem biraz itildiğinde dengeye geri döner mi, yoksa uzaklaşır mı? Aşağı sarkan sarkaç geri döner; tepede dengede duran sarkaç devrilir.
- **Döngü.** Sistem aynı hareketi tekrarlar mı? Kalp atışı ve gezegen yörüngeleri böyledir.
- **Kaos.** Başlangıçtaki çok küçük bir fark zamanla büyük bir farka dönüşür mü? Hava durumu böyledir: kural kesindir, ancak uzun vadeli tahmin yapılamaz. Bu davranışı Lorenz basit bir atmosfer modelinde göstermiştir (Lorenz, 1963).
- **Çatallanma** (*bifurcation*). Bir ayar yavaşça değiştirildiğinde sistemin davranışı aniden değişir mi? Musluk açıldıkça suyun düzgün akıştan çalkantılı akışa geçmesi buna örnektir.

Türleri

- **Sürekli zamanlı.** Zaman kesintisiz akar ve kural diferansiyel denklemle yazılır. Fiziksel sistemler genellikle böyledir.
- **Ayrık zamanlı.** Zaman adım adım ilerler ve kural “bir sonraki adım” olarak yazılır. Bilgisayar simülasyonları ve yıllık nüfus sayımları böyledir.

Niteliksel Bakış

Teorinin ayırt edici özelliği, denklemi tam çözmeye çalışmamasıdır; denklemlerin çoğu zaten kapalı biçimde çözülemez. Bunun yerine davranışın niteliğine bakılır: sistem sakinleşir mi, salınır mı, dağılır mı. Bu bakış on dokuzuncu yüzyılın sonunda Poincaré’nin gök mekaniği çalışmalarıyla başlamıştır (Strogatz, 1994).

Yazılımda Kullanım

Yazılımda bu fikirlerin tümü kullanılmaktadır; yalnızca adları farklıdır. Ortak kalıp şudur:

```
yeniDurum = kural(eskiDurum, olay)
```

Durum Yönetimi

Uygulamanın durumu dağıtık değişkenler yerine tek bir yapıda toplanır. Redux, NgRx ve Elm mimarisi bu yaklaşımı uygular. Kural, yan etkisi olmayan saf bir fonksiyondur ve **reducer** adını alır. Faydası şudur: aynı durum ve aynı olay her zaman aynı sonucu verir; bu nedenle test etmek ve bir hatayı yeniden üretmek kolaylaşır.

Durum Makinesi

Durum uzayını küçültmek hataları azaltır. Üç adet mantıksal değişken (**loading**, **error**, **success**) sekiz kombinasyon verir, ancak bunların yalnızca dördü anlamlıdır. Tek bir alan kullanıldığında geçersiz durumlar hiç oluşmaz:

```
idle | loading | success | error
```

Sipariş akışı, bağlantı yönetimi ve çok adımlı formlar bu yapıya uygundur. Karmaşık sistemlerde iç içe ve paralel durumları ifade etmek için Harel'in durum çizelgeleri (*statecharts*) kullanılır (Harel, 1987).

Event Sourcing ve Snapshot

Son hal yerine olaylar kaydedilir. Kazanç üç yönlüdür: denetim kaydı, geri alma ve geçmişteki bir ana dönüp hatayı inceleme. Oynatma süresini kısaltmak için aralıklarla *snapshot* alınır.

Eşitleme

İki taraf “şuraya kadar gördüm” bilgisini, yani durum vektörünü karşılaştırır ve yalnızca eksik kısım gönderilir. Çevrimdışı çalışan uygulamalar, ortak düzenleme ve veri çoğaltma (*replication*) bu şekilde çalışır.

Tahmin

- *Optimistic update*: sunucu beklenmeden sonuç ekrana yansıtılır.
- Kapasite planlama: yük artışı modellenir ve kaynağın ne zaman dolacağı hesaplanır.
- Oyunlarda ve canlı sistemlerde gecikmeyi gizlemek için bir sonraki konum tahmin edilir.

Kararlılık

Geri beslemeli her altyapı mekanizması bir dinamik sistemdir.

- **Otomatik ölçekleme** (*autoscaling*). Yük artar, sunucu eklenir; yük düşer, sunucu silinir. Ayarlar kötü seçilmişse sistem sürekli açıp kapatarak salınır.
- **Yeniden deneme** (*retry*). Hata alan istemciler tekrar dener, bu yükü artırır ve daha çok hata üretir. Bu kararsız bir döngüdür. Üstel geri çekilme (*exponential backoff*) ve rastgele gecikme (*jitter*) sistemi dengeye geri getirir (Brooker, 2015).
- **Diğerleri**. Hız sınırlama (*rate limiting*), devre kesici (*circuit breaker*) ve TCP tıkanıklık kontrolü aynı mantıkla çalışır (Jacobson, 1988).

Durum Çıkarımı

Ham log ve metriklerden sistemin hali çıkarılır: sağlıklı, yavaşlamış, çökmüş. Sağlık denetimleri (*health check*) ve izleme panelleri bu işi yapar.

Test

Durum uzayı küçük ve açık olduğunda tüm durumlar ve geçişler otomatik olarak gezilebilir. Model denetimi (*model checking*) ve özellik tabanlı test (*property-based testing*) bu fikre dayanır. TLA+ gibi biçimsel belirtim dilleri, dağıtık sistemlerdeki nadir hataları bu yolla bulur (Lamport, 2002).

Sonuç

Bir özelliği tasarlarlarken üç soru sorulabilir: durum tam olarak nedir, hangi olaylar onu değiştirir, hangi durumlar hiç oluşmamalıdır. Bu üç soru, durum uzayı yaklaşımının yazılımdaki en doğrudan karşılığıdır. İlki durum vektörünü, ikincisi geçiş modelini, üçüncüsü durum uzayının sınırını belirler. Sorular cevapsız kaldığında durum koda dağınık ve hatalar “olmaması gereken ama oluşan” kombinasyonlardan çıkar.

Log ile durum arasındaki asimetri bir tasarım kararına dönüşür. Log saklanıyorsa durum her zaman yeniden üretilebilir ve yanlış hesaplanmış bir durum düzeltilebilir. Yalnızca durum saklanıyorsa kaybolan bilgi geri gelmez. Bedeli depolama ve oynatma süresidir; *snapshot* bu bedeli düşürür ama ortadan kaldırmaz.

Çerçevenin sınırları vardır. Durum vektörünün küçük kalması, sistemin Markov özelliğini taşımasına bağlıdır: gelecek yalnızca şimdiye bağlı olmalıdır. Geçmişe uzun süreli bağımlılığı olan sistemlerde ya vektör büyür ya da bilgi kaybedilir. *Transformer* mimarisinin özet vektör yerine tüm geçmişe bakması bu sınırın bir sonucudur. İkinci sınır kaostur: kural kesin olsa bile başlangıç durumundaki küçük ölçüm hataları uzun vadeli tahmini anlamsız kılabilir. Üçüncü sınır çıkarımdadır: hangi değişkenlerin duruma ait olduğu çoğu zaman önceden bilinmez ve yanlış seçim, modelin geri kalanı doğru olsa da tahmini bozar.

Açık kalan soru, öğrenilmiş durum temsillerinin ne kadar güvenilir olduğudur. Elle seçilen bir durum vektörünün her bileşeninin anlamı bellidir. Bir sinir ağının ürettiği gizil durumda ise hangi bilginin tutulduğu ve hangisinin atıldığı doğrudan okunamaz.

Altyapı mekanizmalarını dinamik sistem olarak okumak, kontrol teorisinin kavramlarını (kararlılık, salınım, geri besleme kazancı) yazılım operasyonlarına taşır. Otomatik ölçeklemenin salınması ya da yeniden deneme fırtınası bir kod hatası değil, bir kararlılık sorunudur ve çözümü de kontrol teorisinin araçlarındadır.

Kaynaklar

- Bellman, R. (1957). A Markovian Decision Process. *Journal of Mathematics and Mechanics*, 6(5).
- Brooker, M. (2015). Exponential Backoff and Jitter. *AWS Architecture Blog*.
- Fowler, M. (2005). Event Sourcing. *martinfowler.com*.
- Ha, D. ve Schmidhuber, J. (2018). World Models. arXiv:1803.10122.
- Harel, D. (1987). Statecharts: A Visual Formalism for Complex Systems. *Science of Computer Programming*, 8(3).
- Jacobson, V. (1988). Congestion Avoidance and Control. *ACM SIGCOMM*.

- Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*, 82(1).
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly.
- Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.
- Lorenz, E. N. (1963). Deterministic Nonperiodic Flow. *Journal of the Atmospheric Sciences*, 20(2).
- Shapiro, M., Pregoça, N., Baquero, C. ve Zawirski, M. (2011). Conflict-free Replicated Data Types. *Stabilization, Safety, and Security of Distributed Systems (SSS)*.
- Strogatz, S. H. (1994). *Nonlinear Dynamics and Chaos*. Addison-Wesley.
- Sutton, R. S. ve Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2. baskı). MIT Press.
- Vaswani, A. vd. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*.